

AI 無料枠の課題と限界

AkiNavi HR-AI における AI 利用コスト・無料枠の制約を整理した資料。`ai_logs` テーブルの実績データ (2026 年 5 月) と現行アーキテクチャを照らし合わせる。

大きな変更点 ① (2026-05-19 / コミット `139a4f2`)

メール解析 (`inbound-email`) から AI を完全に除去した。このドキュメントの「実績データ」セクションは AI を使っていた旧構成のものであり、現行構成ではメール処理量に応じて無料枠が枯渇することは起きない。

大きな変更点 ② (2026-05-22 / コミット `b35df40`)

新 Edge Function `match-batch` を導入。ルールベース事前フィルタ + バッチ AI 採点で「1 案件 = 1 AI コール」に圧縮し、`auto-match` も `match-batch` を内部呼び出すように刷新された。その結果、`auto-match` も **Cerebras → Groq 70B → Gemini フォールバック** を持ち、3 段すべて失敗してもルールスコアで全代替される (システムが止まらない)。

1. 現在の AI 構成 (2026-05-23 時点)

場所	AI 使用	モデル	フォールバック
<code>inbound-email</code> メール解析	不使用 (regex + 文章スキャン + <code>skill_master</code> DB 照合のみ)	—	—
<code>match-batch</code> (バッチ採点・MatchingPage 高速/全件 + <code>auto-match</code> 内部呼び出し)	使用	Cerebras <code>llama3.1-8b</code> → Groq <code>llama-3.3-70b-versatile</code> → Gemini <code>gemini-2.5-flash</code>	あり。3 段失敗時はルールスコアで全代替
<code>match-score</code> (UI 単発・duplicate 検出付き)	使用	同上	あり
<code>auto-match</code> (毎朝 JST 9:00 cron)	使用	<code>match-batch</code> を内部呼び出し → 同じ 3 段フォールバック	あり (同上)
<code>poll-email</code> メール種別バッチ分類	使用 (既定 OFF)	Gemini <code>gemini-2.5-flash-lite</code>	なし (失敗時は <code>candidate</code> フォールバック)
ブラウザ (人材・案件登録)	不使用 (Phase 4.11 で「AI で登録」ボタン廃止、登録ボタンは <code>inbound-email</code> regex 経路に一本化)	—	—

各モデルの無料枠

プロバイダ	モデル	無料枠	リセット
Cerebras	llama3.1-8b	実質無制限	—
Groq	llama-3.3-70b-versatile	500K tokens/日	毎日 JST 9:00
Groq	llama-3.1-8b-instant	500K tokens/日	毎日 JST 9:00
Gemini	gemini-2.5-flash / gemini-2.5-flash-lite	プリペイド制（要チャージ）	—

2. 1 回のマッチング処理あたりのトークン消費量

match-batch（バッチ採点・1 案件 1 コール）

ルールベース事前フィルタで上位 topN（既定 10 名）を選び、まとめて 1 リクエストで AI に再採点させるため、案件あたりの AI 呼び出し回数が劇的に減る。

内容	トークン概算
プロンプトヘッダー（指示文 + 150 字 summary 仕様）	約 400 tokens
案件要件（要約済み）	約 500 tokens
人材リスト（topN = 10 名分）	約 2,500 tokens
レスポンス（JSON 配列・各 150 字 summary）	約 1,500 tokens
合計（1 案件 = 10 名分一括）	約 5,000 tokens

match-score（単発・1 ペア）

内容	トークン概算
プロンプトヘッダー（指示文）	約 200 tokens
人材プロフィール（要約済み）	約 500 tokens
案件要件（要約済み）	約 500 tokens
レスポンス（JSON 出力・理由 150 字付き）	約 300 tokens
合計（1 ペア）	約 1,500 tokens

想定処理上限（Groq 70B 単独）

500,000 tokens/日 ÷ 5,000 tokens/案件 ≈ 100 案件/日 (match-batch)
500,000 tokens/日 ÷ 1,500 tokens/ペア ≈ 333 ペア/日 (match-score)

Cerebras 8B が成功すれば Groq を消費しない（実質無制限）。さらに 3 段すべて失敗してもルールスコアで全代替されるため、最悪「AI 採点なし」で進む。

3. 旧構成（AI 解析あり）の実績データ ※歴史的記録

以下は `inbound-email` が AI を使っていた **2026-05-01 ~ 2026-05-16** の `ai_logs` 実績。現行アーキテクチャでは発生しない問題。

日別の処理件数（メール解析 + マッチング）

日付	成功	エラー	合計	状況
5/1	64	14	78	通常
5/9	24	2	26	ポーリング安定化開始
5/10	66	1	67	バックログ消化開始
5/11	420	249	669	バックログ急増
5/12	3,707	755	4,462	ピーク（candidates: 1,518 件登録）
5/13	2,541	5	2,546	バックログ継続
5/14	575	786	1,361	Groq / Gemini 枯渇開始
5/15	22	1,252	1,274	ほぼ全滅（無限リトライ）
5/16	6	3	9	バックログ消化完了

5/15 大量エラーの原因

① Groq が 1 日 125 件（≈ 500K tokens）を処理し終えて 429 を返す

↓

② Gemini にフォールバック

↓

③ Gemini もプリペイドクレジット切れで 429

↓

④ FATAL エラー → ai_logs に type='unknown' として記録

↓

⑤ poll-email が失敗メールを未読に戻す → 5 分後に再取得 → また失敗（無限リトライ）

教訓と現行アーキテクチャでの対処

旧問題	旧 AI 構成での影響	現行アーキテクチャ（コミット <code>139a4f2</code> + <code>b35df40</code> 以降）
添付の長大ファイル	プロンプト 4,300 文字でトランケート → 抽出漏れ	DB 照合方式のため文字数上限なし。長い添付ほど精度向上

旧問題	旧 AI 構成での影響	現行アーキテクチャ（コミット 139a4f2 + b35df40 以降）
同じメールが詰まり続ける	無限リトライ	失敗メールも基本的に保存される（regex が '不明' 等を補完）
Groq 125 件/日上限	バックログ時に枯渇	解析側で AI を使わないため上限なし
Gemini クレジット切れ	全件エラーで詰まる	match-batch は 3 段フォールバック → 最悪ルールスコアで全代替（システム停止なし）
マッチングで AI コールが大量発生	案件×候補者ペアごとに AI コール	match-batch で 1 案件 = 1 AI コール（topN のみ採点）に圧縮

4. メール種別分類（任意機能）のコスト

app_config.email_classify_enabled='true' のときのみ Gemini を消費する。

- 同一受信箱に candidate / project が混在するケース用
- 最大 20 件をまとめて 1 回の API コール（バッチ分類）
- 1 コールあたり概算 1,000～2,000 tokens
- 大半のメールはルールベース事前フィルタ（SKIP*_PATTERNS, PROJECT*_PATTERNS, HR_SUBJECT_PATTERNS）で AI を呼ばずに分類される

実運用ではメール 100 件あたり Gemini コール 1～2 回程度に収まる。

5. ブラウザ側の AI 利用（現状ほぼゼロ）

Phase 4.11（コミット f28ec86）で「AI で登録」ボタンが廃止されたため、人材登録 / 案件登録 UI からは AI を呼ばなくなった。登録ボタンは inbound-email を force=true で叩く経路に一本化されており、regex + skill_master 照合だけで処理される。

src/lib/ai/geminiProvider.ts のコード自体は残っているが、UI からの呼び出し経路は存在しない（将来のための予備）。

6. 現在の課題優先度

課題	深刻度	発生条件	対処
match-batch で 3 段すべて失敗	● 低	Cerebras / Groq / Gemini 同時枯渇	ルールスコアで全代替されるためシステム自体は止まらない。月初に Gemini チャージ・残高アラート設定
match-score で全 AI 枯渇	● 低	1 日マッチング数百ペア超 + Groq + Gemini 同時枯渇	Groq 有料プラン or Gemini 増額。UI 側でリトライ可能
PDF / 画像 未対応	● 中	PDF / 画像のアップロード	テキスト貼り付けを案内

課題	深刻度	発生条件	対処
skill_master の欠落スキル	● 低 (改善中)	業界標準スキル (JP1 等) が抽出されない	コミット <code>acf9d31</code> で 32 件、Phase 4.10 で DWH/工程/IBM 系 32 件追加。 <code>/quality-check</code> スキルで月次レビュー
駅マップの未収載 / 同名衝突	● 低 (改善中)	新規地方の駅名がマップ未登録	<code>[station_unmapped]</code> ログを月次レビュー → 追記

7. 費用試算

Gemini チャージの目安

月間マッチング案件数	Gemini 費用 (概算・ <code>match-batch</code> のみ)
100 案件 (1,000 ペア相当)	\$0.5 程度
1,000 案件 (10,000 ペア相当)	\$3~5
10,000 案件	\$30~50

(バッチ採点により従来の数倍効率化されている)

Groq 有料プラン

モデル	料金	月 1,000 案件マッチでの費用
llama-3.1-8b-instant	\$0.06/1M tokens	約 \$0.3/月
llama-3.3-70b-versatile	\$0.59/1M tokens	約 \$3/月

マッチング処理が天井になる運用なら、Groq 有料プラン (月 \$1~3) を入れておくと安心。

8. 推奨運用

短期: Gemini に少額 (\$10~20) チャージ → `match-batch` / `auto-match` の安定稼働

中期: Groq 70B が枯渇するか観察。週次で `ai_logs` を確認

長期: マッチング数千案件/日が常態化したら Groq 有料 or Gemini 増額

メール取り込み側はもう AI 非依存なので、件数に応じた費用増を気にする必要はない。マッチング側もルールスコアで全代替されるため「止まらない」設計。

更新日: 2026-05-23 / データ出典: `ai_logs` テーブル (Supabase) 旧構成・実績データの保持理由: 5/15 の障害から得た教訓を残すため